

WEB 003 GIN-01 INKKAPLUM SH

说明

本教程中全部文字版教程和代码为 B 站: [InkkaPlum 频道](#) 和知乎: [Inkka Plum](#) 的相关教程所用, 仅供学习。

不得二次用于任何机构/个人再次录制 Go / Gin / Gorm / Redis / MySQL / Vue 或其它任何语言, 框架, 架构, 工具等等教程中。

此外, 下文有任何问题或者需要改进的点, 请联系 UP 主。

前文

而桌面背景和 Code 背景有变化, Code 背景是 Chinozo 的 [桌面背景](#), 而桌面背景是 Harumakigohan 曲绘风格的 PJSK Jump More Jump 内容。

讲解方法和上一次的 Fyne 内容没有什么变化。

UP 的话(前言)

这是一个比较综合的 Go+Gin+Gorm+Redis+MySQL 教程, Up 顾及到了很多基本概念, 因此没有学习过 Go 语言、没有任何 Web 后端开发经验的朋友亦可以学习此内容, 但是之后依然需要花一定的时间学习 Go 基础, 因此建议观看完成之后通过 [本频道\(哔哩哔哩\(Bilibili\): InkkaPlum 频道\)](#) 的 Go 基础视频(两期 Fyne 内容)

具体链接:

<https://www.bilibili.com/video/BV1u142187Ps>

<https://www.bilibili.com/video/BV1kz421i7iB>

强化 Go 语言知识点, 当然不妨将这个视频作为你的你一个 Go 项目。

录制视频不仅是传递知识和解法, 也是对于 Up 自身的一个挑战和知识的再加强, 所以如果有任何不懂的地方, 尤其是视频没有讲清楚的地方, 欢迎私信问 Up 主, Up 主会尽力给出一个可用的解法。

当然, 请关注 Up 的 B 站、知乎、GitHub, 同时多多三连, 当然也可以充电支持 Up 主。大家的支持将给 Up 更多的动力, Up 也会努力给大家带来更好的视频。

同时, 所有课件和代码都在 GitHub 上开源地分享, 如果感到有帮助, 请给一个 Star, 并且别忘了关注 Up 的 Github。

再次注意

本教程中全部文字版教程和代码为 B 站: [InkkaPlum 频道](#) 和知乎: [Inkka Plum](#) 的相关教程所用, 仅供学习。

不得二次用于任何机构/个人再次录制 Go / Gin / Gorm / Redis / MySQL / Vue 或其它任何语言, 框架, 架构, 工具等等教程中。

此外, 下文有任何问题或者需要改进的点, 请联系 UP 主。

必要的基础配置

需要 MySQL,

下载链接如下:

<https://dev.mysql.com/downloads/installer/>

这个配置极其简单, 选第一项即可:

Windows (x86, 32-bit), MSI Installer	8.0.39	2.1M	Download
(mysql-installer-web-community-8.0.39.0.msi)	MD5: d8499da0b2c4b5dfa81a5c5185af9238 Signature		

不要忘记装 MySQL Workbench, 其提供可视化控制台, 可轻松管理 MySQL 环境(类似于我们之前的 MongoDB Compass)。

基础概念

什么是前后端分离

前后端分离的定义

它将传统的 Web 应用从一个整体拆分成两个独立的部分: 前端(Front-end)和后端(Back-end)。

- 前端：主要负责用户界面(UI)的呈现和交互逻辑
- 后端：主要负责业务逻辑、数据处理和与数据库的交互

优点

- 提高开发效率
- 增强系统可维护性
- 提升用户体验
- 技术栈灵活

案例判断

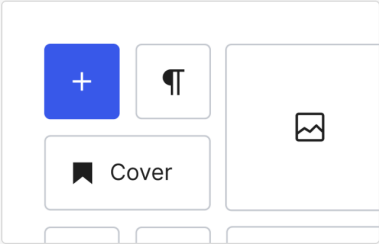
案例 1：Gin+Gorm+Mysql+Vue 博客项目

目录结构: 如: MyBlog-Backend, MyBlog-Frontend。

案例 2: Wordpress简单开发

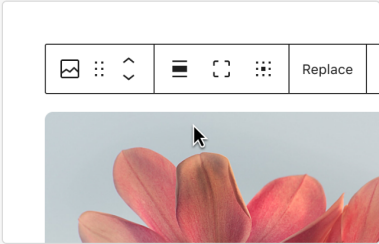
Wordpress 开发

<https://www.wordpress.org>



Оформляйте

Создайте любой сайт с помощью гибких инструментов дизайна и возможностей [блоков](#). Начните с чистого листа или выберите [тему](#). Настройте каждую деталь — код не нужен.



Создавайте

Знайте, как будет выглядеть ваш сайт в режиме реального времени, даже в момент, когда вы создаёте, редактируете или перестраиваете содержимое — с интуитивно понятным редактированием и встроенными функциями для управления всем этим.

[Попробовать редактор блоков](#)



Расширьте

Сделайте из своего сайта всё, что вам требуется. Добавьте магазин, аналитику, новостную ленту, интеграцию с социальными сетями. Вся власть в ваших руках с обширной библиотекой [плагинов](#).

前后端分离中 API 的作用

API 是什么？

API 即为 Application Programming Interface(应用程序接口)。

RESTful API 简述

REST 是什么？

其含义为: Representational State Transfer (译: 表述性状态转移), 其是一种软件架构风格, 而非标准。故 RESTful API 即为一种 REST 风格的接口, 或者说是满足 REST 风格的接口。

在很多年前的案例:

```
http://www.example.com/get_rates?name=usdeur
http://www.example.com/update_rates?...
http://www.example.com/delete_rates?...
#InkkaPlum频道
```

这样设计的 API 并不好, 非常混乱, 但是满足 REST 风格的接口是这样的:

```
GET http://www.example.com/rates
POST http://www.example.com/rates
Data: name=usdeur
...
#InkkaPlum频道
```

只需改变请求方法(Method)就可以完成相关的操作, 易于理解、易于调用。

GET、POST、PUT 和 DELETE

1. GET: 取出资源(一项或多项)
2. POST: 新建一个资源
3. PUT: 更新资源(客户端提供完整资源数据)
4. PATCH: 更新资源(客户端提供需要修改的资源数据)
5. DELETE: 删除资源。

RESTful API 核心要点

1. 资源 (Resources): 每一个 URL 代表一个资源(将互联网的资源抽象成资源, 将获取资源的方式定义为方法), 如 `api/articles/3/`, (以资源为基础, 资源可为一个图片、更多以 JSON 为载体, 如 `{"fromCurrency": "USD", "toCurrency": "KZT", "rate": 479}`)。
2. 使用 HTTP 方法(GET、POST、PUT、DELETE 等)表示对资源的操作。
3. 使用 HTTP 状态码(Http Status Code)表示请求的结果(响应状态码),

如 200(成功)、404(未找到)、500(服务器错误), 以下是更具体情况:

1XX Informational		4XX Client Error Continued	
100	Continue	409	Conflict
101	Switching Protocols	410	Gone
102	Processing	411	Length Required
2XX Success		412	Precondition Failed
200	OK	413	Payload Too Large
201	Created	414	Request-URI Too Long
202	Accepted	415	Unsupported Media Type
203	Non-authoritative Information	416	Requested Range Not Satisfiable
204	No Content	417	Expectation Failed
205	Reset Content	418	I'm a teapot
206	Partial Content	421	Misdirected Request
207	Multi-Status	422	Unprocessable Entity
208	Already Reported	423	Locked
226	IM Used	424	Failed Dependency
3XX Redirectional		426	Upgrade Required
300	Multiple Choices	428	Precondition Required
301	Moved Permanently	429	Too Many Requests
302	Found	431	Request Header Fields Too Large
303	See Other	444	Connection Closed Without Response
304	Not Modified	451	Unavailable For Legal Reasons
305	Use Proxy	499	Client Closed Request
307	Temporary Redirect	5XX Server Error	
308	Permanent Redirect	500	Internal Server Error
4XX Client Error		501	Not Implemented
400	Bad Request	502	Bad Gateway
401	Unauthorized	503	Service Unavailable
402	Payment Required	504	Gateway Timeout
403	Forbidden	505	HTTP Version Not Supported
404	Not Found	506	Variant Also Negotiates
405	Method Not Allowed	507	Insufficient Storage
406	Not Acceptable	508	Loop Detected
407	Proxy Authentication Required	510	Not Extended
408	Request Timeout	511	Network Authentication Required
		599	Network Connect Timeout Error

HTTP STATUS CODES

When a browser requests a service from a web server, an error may occur.
This is a list of HTTP status messages that might be returned.

我们案例中还会用到如 `401(Unauthorized)`: 状态码 `401 Unauthorized(未经授权)`, `201(Created)`: 请求已被成功处理且创建了新的资源。

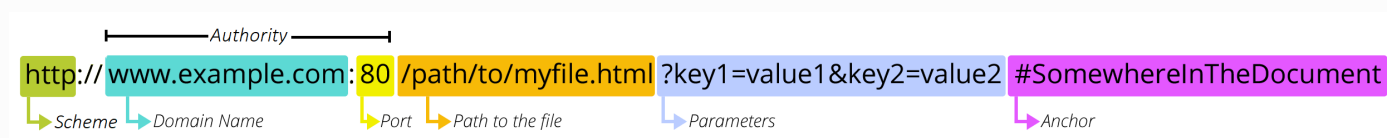
RESTful API 的建议规范

需要注意的是 REST 并没有一个明确的标准, 但一般而言可以有这样的特点, 或者可以称为建议规范:

URL 的重要概念, 其实读一下 MDN Web Docs 即可

https://developer.mozilla.org/zh-CN/docs/Learn/Common_questions/Web_mechanics/What_is_a_URL

`/path/to/myfile.html` 是 Web 服务器上资源的路径。在 Web 的早期阶段, 像这样的路径表示 Web 服务器上的物理文件位置。如今, 它主要是由没有任何物理现实的 Web 服务器处理的抽象。



(图源 MDN Web Docs)

资源路径也就是我们的 Path 这一概念

需要注意的是: URL 是 URI 的一个子集, 这是我们上上一期 **Fyne** 视频就提及到的。

`/ {version} / {resources} / {resource_id}`

例子: `api/v1/articles/3`

1. 使用名词表示资源
2. 使用层次结构
3. 不要使用文件扩展名, 正确使用 `/` 表示层级关系

优势

通过遵循 RESTful API 的建议规范, 可以提高 API 的可读性、可维护性和可扩展性。

其它概念

- 请求地址, 基础地址, (API)接口地址

`请求地址` 在大部分情况下即为 `基础地址` + `api/接口地址`

例子: 基础地址: `http://www.example.com`

接口地址: `/api/v1/user/inkkaplumchannel`

本视频将使用这样的用词。

- 数据格式

JSON, 也是目前主流的

例子:

```
{
  "_id": 1,
  "fromCurrency": "USD",
  "toCurrency": "KZT",
  "rate": 479,
  "date": "2024-08-30T22:45:42.8774003+08:00"
}
//InkkaPlum频道
```

什么是 MVC?

MVC 全称为: Model-View-Controller

MVC 是一种常用的软件架构模式, 旨在将应用程序的关注点分离, 提高代码的可维护性。

假设我们用 Go+gin+gorm+vue 开发了一个简易的博客网站

```
哔哩哔哩InkkaPlum频道的个人博客/
├── backend/
│   ├── main.go
│   └── controllers/
```



```
├── auth.go
├── article.go
├── ...
├── models/
│   ├── user.go
│   ├── article.go
│   └── ...
├── routers/
│   └── router.go
├── config/
│   └── config.yaml
└── frontend/
    ├── src/
    │   ├── App.vue
    │   ├── components/
    │   ├── views/
    │   └── router/
    ├── index.html
    └── ...
```

我们将讨论适用于前后端分离逻辑的MVC内容。

- Model(模型)

例子:

```
models/article.go
```

```
type Article struct {
    Content    string
    CreatedAt  time.Time
    UpdatedAt  time.Time
    ...
}
```

- View(视图)

例子 01:

```
components/ArticleForm.vue
```

```

<template>
  <form @submit.prevent="onSubmit">
    <input type="text" v-model="title" placeholder="标题" />
    <textarea v-model="content" placeholder="内容"></textarea>
    <button type="submit">提交</button>
  </form>
</template>

<script>
...
</script>

```

例子 02:

```

<h2>{{ article.Title }}</h2>
<p>{{ article.Preview }}</p>

```

- Controller(控制器)

例子 01:

```
controllers/article.go
```

```

func CreateArticleHandler(c *gin.Context) {
    var article models.Article
    if err := c.ShouldBindJSON(&article); err != nil {
        // ... 处理错误
    }
    //...
    //InkkaPlum频道

    c.JSON(http.StatusCreated, article)
}

```

我们的例子中, MVC 的工作流程

1. 用户对界面进行操作, 如点击点赞按钮
2. View 感知这些事件, 通知 Controller 进行处理(`POST`)
3. Controller 处理相关业务, 对 Model 的业务数据进行更新

4. View 更新用户界面, 赞+1,

快速记忆: 用户操作-->View -->Controller-->Model-->View。MVC 通信过程都是单向的。

前端路由和后端路由

代码例子: `api.GET("/articles/:id/like", controllers.GetArticleLikes)`

- 用户在浏览器中输入一个 URL, 前端路由根据这个 URL 更新页面内容。如: `{ path: '/register', name: 'Register', component: Register }`, 如 `http://www.example.com/register` 或 `http://www.example.com/articles/2` 等等。
- 在这个页面中, 代码如有涉及到如 `await axios.get<Article>(`/articles/${id}`)`

Gin 和 Gorm 介绍

Go 语言在 Web 后端开发使用广泛, 而其中 Gin 和 Gorm 是非常有名的。

Gin 框架

Gin 是一个使用 Go 语言开发的 Web 框架。

主要特点:

- 高性能
- 中间件支持
- 路由分组

Gorm

Gorm 是 Golang 中最流行的 ORM (对象关系映射) 库(ORM Library)

什么是 ORM?

ORM - Object Relational Mapping。

优势:

- 简单易用
- 自动迁移
- 支持多种数据库

Gin 和 Gorm 结合使用

Gin 和 Gorm 经常一起使用来构建 Go Web 应用程序。

正式开始写代码

活用 Viper 读取配置文件

`.yaml` 文件介绍:

YAML 是"YAML Ain't a Markup Language"(YAML 不是一种标记语言)的递归缩写。在开发的这种语言时,YAML 的意思其实是: "Yet Another Markup Language"(仍是一种标记语言),但为了强调这种语言以数据为中心,而不是以标记语言为重点,故改变名字。

例子:

```
receipt: InkkaPlum Channel
date: 2024
```

我们案例中要使用此文件记录关键信息。

新建一个文件夹,叫 `config`, 内部创建一个 `config.go` 文件, 并且再创建一个 `config.yaml` 文件, 内部先这样写。

```
app:
  name: CurrencyExchangeApp
  port: :3000

database:
  host: localhost
  port: :3306
  user: your_username
  password: your_password
  name: currency_exchange_db
```

敲命令:

```
go mod init
```

```
go get -u github.com/gin-gonic/gin
go get github.com/spf13/viper
```

结构体

```
type Config struct {
    App struct {
        Name string
        Port string
    }
    Database struct {
        Host      string
        Port      string
        User      string
        Password  string
        Name      string
    }
}
```

Go 基础小提示:

`%v` 占位符: 相应值的默认格式

Viper 官方文档重要内容

官方文档: <https://github.com/spf13/viper>

```
/*
Example config:

module:
    enabled: true
    token: 89h3f98hbwf987h3f98wenf89ehf
*/
type config struct {
    Module struct {
        Enabled bool

        moduleConfig `mapstructure:",squash"`
    }
}

// moduleConfig could be in a module specific package
type moduleConfig struct {
    Token string
}

var C config

err := viper.Unmarshal(&C)
if err != nil {
    t.Fatalf("unable to decode into struct, %v", err)
}
```

上手 Gin

<https://gin-gonic.com/zh-cn/docs/quickstart/>

```
package main

import "github.com/gin-gonic/gin"

func main() {
    r := gin.Default()
    r.GET("/ping", func(c *gin.Context) {
        c.JSON(200, gin.H{
            "message": "pong",
        })
    })
    r.Run() // listen and serve on 0.0.0.0:8080
}
```

可以用 `Air` 工具避免不停的 `ctrl+c` 然后 `go run .`, 但本视频将不采用 `Air` 工具。

Router 路由的基本配置

创建文件夹 `router`, 新建必要文件 `router.go`。

官方文档案例:

<https://gin-gonic.com/zh-cn/docs/examples/http-method/>

```
func main() {
    // 禁用控制台颜色
    // gin.DisableConsoleColor()

    // 使用默认中间件 (logger 和 recovery 中间件) 创建 gin 路由
    router := gin.Default()

    router.GET("/someGet", getting)
    router.POST("/somePost", posting)
    router.PUT("/somePut", putting)
    router.DELETE("/someDelete", deleting)
    router.PATCH("/somePatch", patching)
    router.HEAD("/someHead", head)
    router.OPTIONS("/someOptions", options)

    // 默认在 8080 端口启动服务, 除非定义了一个 PORT 的环境变量。
```

```
router.Run()  
// router.Run(":3000") hardcode 端口号  
}
```

本项目案例如下:

登录, 注册:

`/api/auth/login` 以及 `/api/auth/register`

文章:

`/api/articles`

路由组问题

那么在 Gin 的官方文档内, 已经为我们提供了解法

<https://gin-gonic.com/zh-cn/docs/examples/grouping-routes/>

```
func main() {  
    router := gin.Default()  
  
    // 简单的路由组: v1  
    v1 := router.Group("/v1")  
    {  
        v1.POST("/login", loginEndpoint)  
        v1.POST("/submit", submitEndpoint)  
        v1.POST("/read", readEndpoint)  
    }  
  
    // 简单的路由组: v2  
    v2 := router.Group("/v2")  
    {  
        v2.POST("/login", loginEndpoint)  
        v2.POST("/submit", submitEndpoint)  
        v2.POST("/read", readEndpoint)  
    }  
  
    router.Run(":8080")  
}
```



```
}
```

Go 基础概念 - Map 的重要提示

定义: Map 是一种无序的键值对的集合。

下面是一些基本例子:

```
m := make(map[string]int)

m := make(map[string]int, 10)
```

```
// 使用字面量创建 Map
m := map[string]int{
    //键值对 - key: value
    "Harumakigohan": 10,
    "Chinozo": 5,
    "Daibakuhasin": 3,
}
```

```
//活用例
v1 := m["Harumakigohan"]
v2, ok := m["DECO27"]
```

`gin.H` 源代码:

```
type H map[string]any
```

`any` 的说明

```
type any = interface{}
```

利用案例:

```
m := map[string]any{
    //键值对 - key: value
    "Harumakigohan": "10",
    "Chinozo": "グッバイ宣言",
    "Daibakuhasin": 3,
}
```

```
package main

import "fmt"

func main() {
    m := map[string]int{
        //键值对 - key: value
        "Harumakigohan": 10,
        "Chinozo":      5,
        "Daibakuhasin": 3,
    }

    //活用例
    v1 := m["Harumakigohan"]
    v2, ok := m["DECO27"]

    fmt.Println(v1)
    fmt.Println(v2, ok)
}
```

如果配置文件中未对端口进行设定的对策

```
if port == ""{
    port = ":8080"
}
```

Gorm+ Mysql 部分

官方文档(中文)

https://gorm.io/zh_CN/docs/

GORM 为什么成为了非常主流的选择？

1. 简洁的 API 设计
2. 强大的功能
3. 社区活跃

基本配置:

```
go get -u gorm.io/gorm
```

官方案例

```
import (  
    "gorm.io/driver/mysql"  
    "gorm.io/gorm"  
)  
  
func main() {  
    // 参考 https://github.com/go-sql-driver/mysql#dsn-data-source-name 获取详情  
    dsn := "user:pass@tcp(127.0.0.1:3306)/dbname?  
charset=utf8mb4&parseTime=True&loc=Local"  
    db, err := gorm.Open(mysql.Open(dsn), &gorm.Config{})  
}  
  
//db *gorm.DB
```

其它配置: 打开数据库连接的空闲个数、最大连接个数以及其它一些内容。

官方文档

```
// SetMaxIdleConns 用于设置连接池中空闲连接的最大数量。  
sqlDB.SetMaxIdleConns(10)  
  
// SetMaxOpenConns 设置打开数据库连接的最大数量。  
sqlDB.SetMaxOpenConns(100)  
  
// SetConnMaxLifetime 设置了连接可复用的最大时间。  
sqlDB.SetConnMaxLifetime(time.Hour)
```

global.go 重要说明

案例

```
var (  
    Logger    *logrus.Logger  
    Db        *gorm.DB  
)
```

实现注册功能

我们将实现注册功能。

模型相关问题的解决

https://gorm.io/zh_CN/docs/models.html

GORM 通过将 Go 结构体(Go structs) 映射到数据库表来简化数据库交互。

GORM 提供了一个预定义的结构体, 名为 `gorm.Model`, 其中包含常用字段:

```
// gorm.Model 的定义  
type Model struct {  
    ID          uint           `gorm:"primaryKey"`  
    CreatedAt   time.Time  
    // 在创建记录时自动设置为当前时间  
    UpdatedAt   time.Time  
    //每当记录更新时, 自动更新为当前时间  
    DeletedAt   gorm.DeletedAt `gorm:"index"`  
    //用于软删除  
}
```

可直接在结构体中嵌入 `gorm.Model`, 以便自动包含这些字段。

这对于在不同模型之间保持一致性并利用 GORM 内置的约定非常有用。

最终案例:

```
{
  "username": "your_username",
  "password": "your_password"
}
```

字段标签官方文档

https://gorm.io/zh_CN/docs/models.html

GORM 的约定

根据约定, 默认地, GORM 会使用 ID 作为表的主键。

与此同时, 数据表的列名使用的是 struct 字段名的蛇形命名(Snake Case),

案例:

CreatedAt => created_at

```
type User struct {
  ID          uint           // 列名是 `id`
  Name        string          // 列名是 `name`
  Birthday    time.Time       // 列名是 `birthday`
  CreatedAt   time.Time       // 列名是 `created_at`
}
```

注册最终代码

```
func Register(ctx *gin.Context) {
  var user models.User
  if err := ctx.ShouldBindJSON(&user); err != nil {
    ctx.JSON(http.StatusBadRequest, gin.H{"Error!": err.Error()})
    return
  }

  hashedPwd, err := utils.HashPassword(user.Password)

  if err != nil {
    ctx.JSON(http.StatusBadRequest, gin.H{"Error!": err.Error()})
    return
  }
}
```

```

}

user.Password = hashedPwd

token, err := utils.GenerateJWT(user.Username)

if err != nil {
    ctx.JSON(http.StatusInternalServerError, gin.H{"error":
err.Error()})
    return
}

if err := global.DB.AutoMigrate(&user); err !=nil{
    ctx.JSON(http.StatusInternalServerError, gin.H{"error":
err.Error()})
}

if err := global.DB.Create(&user).Error; err != nil {
    ctx.JSON(http.StatusInternalServerError, gin.H{"error":
err.Error()})
    return
}

ctx.JSON(http.StatusOK, gin.H{"token": token})
}

```

Gin 案例代码:

```

func getBook(c *gin.Context) {
    bookID := c.Param("id") // Extracting parameter from URL
    c.JSON(200, gin.H{"book_id": bookID}) // Sending JSON
response
}

```

将请求体绑定到结构体中, 需要活用模型绑定。

案例:

```

type Login struct {
    User      string `form:"user" json:"user" xml:"user"
binding:"required"`
}

```

```

    Password string `form:"password" json:"password"
xml:"password" binding:"required"`
}

func main() {
    router := gin.Default()

    // 绑定 JSON ({"user": "manu", "password": "123"})
    router.POST("/loginJSON", func(c *gin.Context) {
        var json Login
        if err := c.ShouldBindJSON(&json); err != nil {
            c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        }
        return

        if json.User != "manu" || json.Password != "123" {
            c.JSON(http.StatusUnauthorized, gin.H{"status":
"unauthorized"})
            return
        }
        c.JSON(http.StatusOK, gin.H{"status": "you are logged in"})
        ...
    })
}

```

<https://gin-gonic.com/zh-cn/docs/examples/binding-and-validation/>

利用 `ShouldBindJSON` 方法。

*JSON部分和模型部分大小写可不一致: `UseRName` , `username` 是可以的, 但字母不可不一致 `UzerNaam` 则完全不可。

```

    if err := c.ShouldBindJSON(&user); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error":
err.Error()})
        return
    }
}

```

官方文档说明:

Type - Should bind

Methods

Behavior - 这些方法属于 ShouldBindWith 的具体调用。 如果发生绑定错误, Gin 会返回错误并由开发者处理错误和请求。

请求头, 请求体相关内容

http 请求报文包含三个部分(请求行 + 请求头 + 请求体)

请求行包含三个内容: method(如 GET/POST 等) + request-URI(如 <http://www.example.com/api/v1/test>) + HTTP-version(如 `HTTP/1.1`)

请求头(Request Header)

MDN 文档:

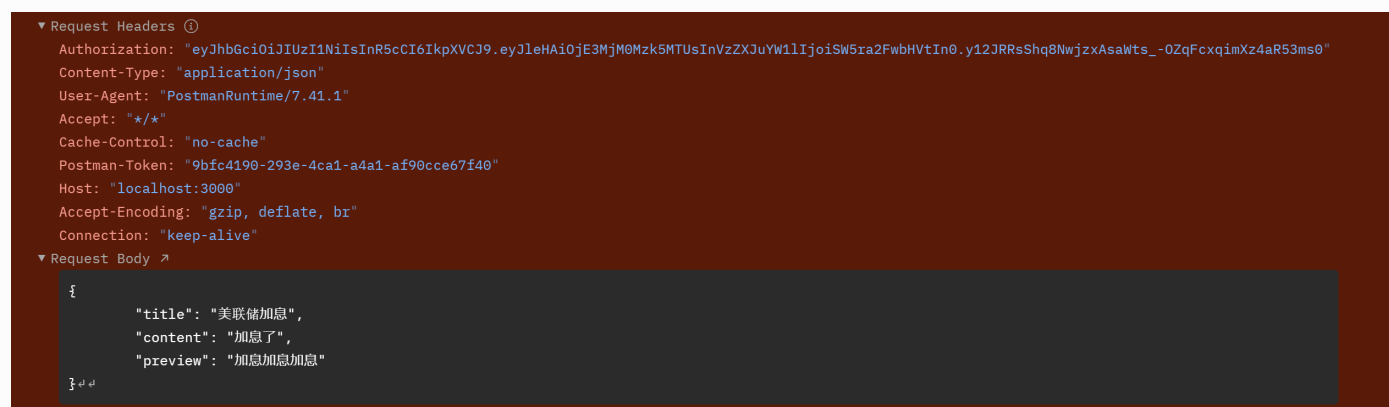
https://developer.mozilla.org/zh-CN/docs/Glossary/Request_header

请求头由 key/value 对 也就是键值对(kv 对)组成, 每行为一对, key 和 value 间通过冒号 (:) 分割。

常见的请求字段

如 `Authorization`, 用于对应的认证信息。

Postman 也可以看到这些重要内容:



请求体(Request Body)

我们写得 JSON 内容, 发送给服务器的数据。

响应也由行、头、体这个概念

Postman 也可以看到这些重要内容:

Body	Cookies	Headers (3)	Test Results	 401 Unauthorized	32 ms	158 B	 Save as example	ed
	Key			Value				
	Content-Type				application/json; charset=utf-8			
	Date				Fri, 30 Aug 2024 13:42:09 GMT			
	Content-Length				25			

状态行(HTTP-version+状态码+状态码的文本描述): 例子: HTTP/1.1 200 OK

响应头: 如: Date 标头: 消息产生的时间

响应体: 当 Web 服务器接收到 Web 客户端的请求报文后, 对 HTTP 请求报文进行解析, 将 Web 客户端的请求的对象取出打包, 通过 HTTP 响应报文将数据传回给客户端; 若出现错误, 则返回包含对应错误的错误代码和错误原因。

加密逻辑- Bcrypt Package

Bcrypt 是一种用于密码哈希的加密算法，它是基于 Blowfish 算法的加强版，被广泛应用于存储密码和进行身份验证。

优势

- 安全性高: Bcrypt 采用了 `salt` 和 `cost` 两种机制,可有效地防止彩虹表攻击和暴力破解攻击,从而保证安全性。

\$2a\$12\$R9h/cIPz0gi.URNNX3kh2OPST9/PgBkqquzi.Ss7KIUgO2t0jWMUW
__/_/_ _____/ _____/_
Alg Cost Salt Hash

案例:

```
import "golang.org/x/crypto/bcrypt"
func main() {
    password := "123456"
    hash, err := bcrypt.GenerateFromPassword([]byte(password),
    bcrypt.DefaultCost)
    ...
    // DefaultCost int = 10
}
```

import 部分

```
import(
    "golang.org/x/crypto/bcrypt"
)
```

```
go get -u golang.org/x/crypto/bcrypt
```

JWT 定义

官网

<https://jwt.io/introduction>

JSON Web Token(JWT | json 网络令牌)是一种开放标准(RFC 7519), 用于在网络应用环境间安全地传递声明(claims)。JWT 是一种紧凑且自包含的方式, 用于作为 JSON 对象在各方之间安全地传输信息。由于其信息是经过数字签名的, 所以可以确保发送的数据在传输过程中未被篡改。

JWT 案例:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJpbmtrYXBsdW1jaG
FubmVsIiwiaXhwIjoiaWTE0NTE0In0.-
xET51cCeonNbsZlT0IB0rZruoj37kSOW4FZu_bnPgq
```

组成部分:

- Header

- Payload
- Signature

`xxxxx.yyyyy.zzzzz` 分别对应上面三个部分。

Header 案例

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

进行 base64 加密(可解密),构成了第一部分 `eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9`

Payload 案例

Payload 部分包含所传递的声明(Claims)。

有三种:

注册声明: 这些声明是预定义的, 非必须使用的但被推荐使用。官方标准定义的注册声明

如 `exp`, 即为过期时间

公共声明: JWT 签发方可以自定义的声明

如 `username`

私有声明: JWT 签发方因为项目需要而自定义的声明, 更符合实际项目场景, 区别于 `注册声明` 和 `公共声明`。

然后将其进行 Base64 加密, 得到 JWT 的第二部分。

Signature 简述

用于验证消息在此过程中没有更改。

用法

请求头里加入 Authorization, 并加上 Bearer 前缀。

可用的库:

<https://jwt.io/libraries>

```
go get github.com/golang-jwt/jwt/v5
```

过期时间解释



是 Unix epoch

CRUD 简述

代表 Create(创建)、Read(读取)、Update(更新)和 Delete(删除)。

在计算机程序设计中，CRUD 是对数据进行的一系列基本操作

准备 自动迁移

`AutoMigrate` 会创建表

```
db.AutoMigrate(&User{})
```

GORM Create 部分

GORM 官方文档 Create 部分

https://gorm.io/zh_CN/docs/create.html

```
user := User{Name: "Jinzhu", Age: 18, Birthday: time.Now()}

result := db.Create(&user) // 通过数据的指针来创建

user.ID           // 返回插入数据的主键
result.Error       // 返回 error
result.RowsAffected // 返回插入记录的条数
```

实现登录功能

我们将实现登录功能

结构体标签

定义：

在结构体中, 在成员后面使用 ```, 定义一些描述性信息, 这就叫 `tag``, 也就是标签。

```
var input struct {
    Username string `json:"username"`
    Password string `json:"password"`
}
```

由一个或者多个键值对组成。

```
key1:"value1" key2:"value2" key3:"value3"...
```

具体作用

标签的常见用途包括控制结构体字段在序列化、反序列化、数据库操作等过程中的行为。

将结构体转换为 JSON 时

`Username` 字段会被转换为 `"username"`, 例如：

```
input := struct {
    Username string `json:"username"`
    Password string `json:"password"`
}{}

```

```

        Username: "B站InkkaPlum频道",
        Password: "114514",
    }

    jsonData, _ := json.Marshal(&input)

    strJsonData := string(jsonData)

    fmt.Println(strJsonData)

    //输出: {"username":"B站InkkaPlum频道","password":"114514"}

```

```

jsonData := `{"username":"InkkaPlum频道","password":"114514"}`

var input struct {
    Username string `json:"username"`
    Password string `json:"password"`
}

_ = json.Unmarshal([]byte(jsonData), &input)

fmt.Println(input)

```

前端部分:

```

interface ExchangeRate {
    fromCurrency: string;
    toCurrency: string;
    rate: number;
}

```

需要更改。

假如不写:

```
{
  "ID": 7,
  "FromCurrency": "USD",
  "ToCurrency": "RUB",
  "Rate": 87,
  "Date": ""
}
```

假如写了:

```
{
  "_id": 8,
  "fromCurrency": "USD",
  "toCurrency": "KZT",
  "rate": 479,
  "date": ""
}
```

<https://github.com/gin-gonic/gin/issues/3570>

GORM 进阶

查看官方文档即可: https://gorm.io/zh_CN/docs/query.html

```
// Get first matched record
db.Where("name = ?", "jinzhu").First(&user)
// SELECT * FROM users WHERE name = 'jinzhu' ORDER BY id LIMIT
1;
```

验证密码

最后代码:

```
func CheckPassword(password, hash string) bool {
  err := bcrypt.CompareHashAndPassword([]byte(hash),
    []byte(password))
  return err == nil
}
```

auth_controller.go

```
if !utils.CheckPassword(input.Password, user.Password) {
    c.JSON(http.StatusUnauthorized, gin.H{"error": "Invalid
credentials"})
    return
}
```

router 部分更改:

```
auth := r.Group("/api/auth")
{
    auth.POST("/login", controllers.Login)
    auth.POST("/register", controllers.Register)
}
```

测试 JSON

```
{
  "username": "inkkaplum123456",
  "password": "123456"
}
```

汇率兑换功能

exchange_rate_controller.go

最终案例:

```
{
  "_id": "",
  "fromCurrency": "EUR",
  "toCurrency": "USD",
  "rate": 1.1,
  "date": "",
},
```

返回案例:

```
[
  {
    "_id": 1,
    "fromCurrency": "EUR",
    "toCurrency": "USD",
    "rate": 1.1,
    "date": ""
  },
  {
    "_id": 2,
    "fromCurrency": "USD",
    "toCurrency": "JPY",
    "rate": 147,
    "date": ""
  }
]
```

结构体切片的提示

```
type prices struct{
    Name string
    CurrentPrice int
}

var Prices []prices = []prices{
{
    Name: "Sberbank",
    CurrentPrice: 3000,
},
{
    Name: "Yandex",
    CurrentPrice: 200,
},
}

fmt.Println(Prices)
// [{Sberbank 3000} {Yandex 200}]
fmt.Println(Prices[0])
// {Sberbank 3000}
```

大小是动态的

```
Prices = append(Prices, prices{
    Name: "Mailru",
    CurrentPrice : 150,
})
fmt.Println(Prices)
// [{Sberbank 3000} {Yandex 200} {Mailru 150}]
```

GORM 文档

```
db.Find(&users, []int{1,2,3})
// SELECT * FROM users WHERE id IN (1,2,3);
```

中间件

要求:

用户必须要登录或者注册, 有了 jwt 才可以创建对应的汇率内容。

中间件是为了过滤路由而发明的一种机制, 也就是 http 请求来到时先经过中间件, 再到具体的处理函数。

- 请求到到达我们定义的处理函数之前, 拦截请求并进行相应处理(比如: 权限验证, 数据过滤等), 这个可以类比为前置拦截器或前置过滤器;

自定义中间件, 请参考 Gin 的官方文档 - <https://gin-gonic.com/zh-cn/docs/examples/custom-middleware/>

```
func Logger() gin.HandlerFunc {
    return func(c *gin.Context) {
        t := time.Now()

        // 设置 example 变量
        c.Set("example", "12345")

        // 请求前

        c.Next()

        // 请求后
```

```

latency := time.Since(t)
log.Print(latency)

// 获取发送的 status
status := c.Writer.Status()
log.Println(status)
}

func main() {
    r := gin.New()
    r.Use(Logger())

    r.GET("/test", func(c *gin.Context) {
        example := c.MustGet("example").(string)

        // 打印: "12345"
        log.Println(example)
    })

    // 监听并在 0.0.0.0:8080 上启动服务
    r.Run(":8080")
}

```

`gin.HandlerFunc` 内容。

```
type HandlerFunc func(*Context)
```

在 `router.go` 内

```

router = gin.Default()
router.Use(middlewares.MyMiddleware())

```

验证 JWT

案例:

```
"Bearer your_token"
```

StackOverflow 的答案

<https://stackoverflow.com/questions/45405626/how-to-decode-a-jwt-token-in-go>

参考答案:

```
// For example to parse a JWT with HMAC verification.
tokenString := /* raw JWT string*/

token, err := jwt.Parse(tokenString, func(token *jwt.Token)
(interface{}, error) {
    if _, ok := token.Method.(*jwt.SigningMethodHMAC); !ok
{
    return nil, errors.New("unexpected signing method")
}
    return []byte(/* your JWT secret*/), nil
})
if err != nil {
    // handle err
}

// validate the essential claims
if !token.Valid {
    // handle invalid token
}
```

源代码内容:

```
type MapClaims map[string]interface{}
```

Set 方法

Set 用于存储此上下文专用的新键值对, 如

```
ctx.Set("username", username)
```

Set 及 Get 逻辑参考:

```
c *gin.Context -> func1 -> func2(c.Set(key,value)) => func3(c.Get(key))
```

这样后续的处理函数可以通过 `c.Get("username")` 访问这个值。

`ctx.Next()` 的作用

调用下一个中间件或处理器

如果 `c.Next()` 之前有 `c.Abort()`, 后续的处理将会被终止, 不会继续执行

案例代码

```
package main

import (
    "fmt"
    "net/http"

    "github.com/gin-gonic/gin"
)

func fun1(c *gin.Context) {
    fmt.Println("func1 start")
    fmt.Println("func1 end")
}
func fun2(c *gin.Context) {
    fmt.Println("func2 start")
    fmt.Println("func2 end")
}
func fun3(c *gin.Context) {
    fmt.Println("func3 start")
    fmt.Println("func3 end")
}

func main() {
    r := gin.Default()

    r.Use(fun1, fun2, fun3)

    // Example ping request.
    r.GET("/ping", func(c *gin.Context) {
        fmt.Println("ping start")

        c.String(http.StatusOK, "pong")
    })
}
```

```
    fmt.Println("ping end")
})

r.Run(":8080")
}
```

```
//输出
func1 start
func1 end
func2 start
func2 end
...
ping start
ping end
```

用了 `c.Next()` 后

```
func fun1(c *gin.Context) {
    fmt.Println("func1 start")
    c.Next()
    fmt.Println("func1 end")
}
func fun2(c *gin.Context) {
    fmt.Println("func2 start")
    c.Next()
    fmt.Println("func2 end")
}
func fun3(c *gin.Context) {
    // ...
}
//输出
func1 start
func2 start
func3 start
ping start
ping end
func3 end
func2 end
func1 end
```

如有 `c.Abort`:

```
func fun1(c *gin.Context) {
    fmt.Println("func1 start")
    c.Abort()
    c.Next()
    fmt.Println("func1 end")
}
func fun2(c *gin.Context) {
    // ...
}
func fun3(c *gin.Context) {
    // ...
}

//输出:
func1 start
func1 end
```

路由部分最终配置

```
api.Use(middlewares.AuthMiddleware())
```

测试案例

```
{
    "fromCurrency": "EUR",
    "toCurrency": "USD",
    "rate": 1.1
}
```

文章逻辑

```
type Article struct {
    gorm.Model
    Title      string
    Content    string
    Preview    string
    Likes      int `gorm:"default:0"` // 点赞数，默认为0
}
```

使用 `global.DB.Create(&article)` 将新文章插入到数据库中。

前端部分提示

```
export interface Article {  
  ID: string;  
  Title: string;  
  Preview: string;  
  Content: string;  
}
```

```
const response = await axios.get<Article>(`/articles/${id}`);
```

案例

<http://localhost:3000/api/articles/1>

路由参数

静态和参数路由概念:

1. 静态路由: 完全匹配的路由, 如 `/articles`。
2. 参数路由: 在路径中带上了参数(Params)的路由, 如 `/articles/:id`。

在 Gin 中, `c.Param` 方法可以获取路径中的参数, 那么其会返回 URL 参数的值, 比如说 1。

Gorm, 找到第一条匹配的记录

```
db.Where("name = ?", "jinzhu").First(&user)
```

测试用:

```
{  
  "Title": "欧央行加息了!",  
  "Preview": "加息!",  
  "Content": "加息, 重要内容"  
}
```


Redis 相关重要概念

<https://redis.io/> Redis 官网

安装 Redis

<https://github.com/tporadowski/redis/releases>

选择 `Redis-x64-5.0.14.1.msi` 即可

定义: Redis 是一个高性能 NoSQL 的, 用 C 实现, 可基于内存亦可持久化的 Key-Value 数据库(键值对存储数据库), 并提供多种语言的 API。

根据月度排行网站 `DB-Engines.com` 的数据, Redis 是现在最受欢迎的 NoSQL 数据库之一 (MongoDB 亦为 NoSQL 数据库)。

与 MySQL 数据库不同的是(为了实现数据的持久化存储, Mysql 将数据存储到了磁盘中), Redis 的数据是存在内存中的。它的读写速度非常快。

一些基础概念:

Redis 有如字符串(String)、列表(List)、集合(Set)、有序集合(ZSet/Sorted Set)这样的基础数据类型。

优势:

性能好、丰富的数据类型。

持久化

定义: 持久化是指将数据写入持久存储(durable storage), 如固态硬盘(SSD)。

Redis 提供了一系列选项。

1. RDB(Redis Database): RDB 持久化通过在指定的时间间隔内创建数据集的快照来保存数据。

2. AOF(Append Only File): AOF 持久化记录服务器接收到的每一个写操作，并将这些操作追加到日志文件中。
3. 无持久化: 完全禁用 Redis 的持久化机制, 这意味着 Redis 只会在内存中存储数据。
4. AOF + RDB 组合: 可以在同一个实例中同时启用 RDB 和 AOF 持久化。

设置方法

在 redis 目录下, 找到 `redis.windows.conf` 即可。

RDB 部分:

默认情况下, Redis 会将数据集的快照保存在磁盘上一个名为 `dump.rdb` 的二进制文件中。

目录(一般情况)

`C:\Program Files\Redis`

在 `redis.windows.conf` 文件内

找到

```
save 60 1000
```

AOF 部分

找到

将 `no` 改为 `yes`, 开启 AOF

```
appendonly yes
```

它在我们的项目中的作用

典型的案例

如一个帖子:

B站InkkaPlum频道 11:13

最近在跟着做Lab, 收获很多!

点赞114 转发5 评论14

InkkaPlum频道

08月24日

午夜敲代码小感想

Up到了晚上, 常喜欢边做题目边听些电音, 或边敲代码边听电音, 每次听到比如说"It Ain't Me", "Silhouettes", 或者是Martin Garrix这一类欧陆的制作人写得曲子, 往往有一种在奥斯陆、阿姆斯特丹这一类欧陆城市写代码的感觉, 仿佛刚刚从Tomorrowland回来, 窗外就是Oslo Sentral。



转发

5

14

这个帖子下面的点赞、转发、评论数内容都是一个典型的案例, 在我们的案例中就是点赞功能。

Go-redis 定义: Go-redis 是 Golang 中用于与 Redis 交互的强大工具, 支持 Redis Server 的 Golang 客户端。

GitHub: <https://github.com/redis/go-redis>

文档: <https://redis.uptrace.dev/zh/>

```
go get -u github.com/go-redis/redis
```

```
redis.go
```

```
package config

import (
    "github.com/go-redis/redis"
)

var RedisClient *redis.Client

func InitRedis() {
    RedisClient = redis.NewClient(&redis.Options{
        Addr: "localhost:6379",
        DB:   0, // 默认数据库(use default DB)
        Password: "",
    })

    _, err := RedisClient.Ping().Result()
    if err != nil {
        panic("Failed to connect to Redis")
    }
}
```

`localhost:6379`——在安装过程中设置的端口。

默认情况下就是 6379, 截屏参考

Port to run Redis on:

6379

global.go

```
var (  
    DB *gorm.DB  
    RedisDB *redis.Client  
)
```

实现点赞功能

like_controller.go

给文章增加点赞数

redis key 命名规范的设计

key 单词与单词之间以 (:)分割, 如 `user:userinfo`, `article:1:likes`

SET,INCR,DECR,GET命令

`SET` 命令用于设置给定 key 的值。

`INCR` 将 key 中储存的数字值增一。

若 key 不存在,那么 key 的值会先被初始化为 0,然后再执行操作。(0 -> 1)

`DECR` 则将 key 中储存的数字减一。

`GET` 命令用于获取指定 key 的值。

Go-redis活用:

```
// 执行 Redis 命令:
val, err := rdb.Get("key").Result()
fmt.Println(val)
```

重要概念: 原子性

原子性确保一个操作在执行过程中是不可被打断的。对于原子操作, 要么所有的步骤都成功完成并对外可见, 要么这些步骤都不执行, 系统的状态保持不变。

`INCR` 和 `DECR` 操作在 Redis 中就是原子操作(原子性的)。

例子: 用户 A 发起点赞请求, Redis 执行 `INCR` 命令, 将点赞数从 10 增加到 11。

用户 B 几乎同时发起点赞请求, Redis 执行 `INCR` 命令, 将点赞数从 11 增加到 12。

因此, 若多个操作只是单纯的对数据进行增加或减少值, Redis 提供的 `INCR` 和 `DECR` 命令可以直接帮助我们进行并发控制。

但是, 若多个操作不只是单纯的进行数据增减值, 还包括更复杂的操作, 如: 逻辑判断, 此时 Redis 的单命令操作无法保证多个操作互斥执行, 故可用 Lua 脚本来解决此问题, 本教程不涉及这一点, 以后会有专门的分析。

路由部分最后配置

`router.go`

```
api.POST("/articles/:id/like", controllers.LikeArticle)
api.GET("/articles/:id/like", controllers.GetArticleLikes)
```

到此, 我们后端部分严格来说就算彻底完成了。

测试用:

`GET`

```
http://localhost:3000/api/articles/1/like
```

可以看到, 正确地得到了 likes 为 0 的响应内容。

增加前端功能

如果是直接用 GitHub 对应课件上面得到的前端内容, 无需进行任何更改、添加

如果是继续基于上一次本频道(B 站: InkkaPlum 频道)的Vue 教程的对应源码进行的后端部分学习, 则需按下面内容进行更改。

在 `article.d.ts` 中, 添加。

```
export interface Like {  
  likes: number;  
}
```

在 `NewsDetailView.vue` 中, 修改并添加代码

修改:

```
import type { Article, Like } from "../types/Article";  
  
// 而非原来的  
import type { Article } from "../types/Article";
```

添加:

```
const likeArticle = async () => {  
  try {  
    const res = await axios.post<Like>(`articles/${id}/like`);  
    likes.value = res.data.likes;  
    await fetchLike();  
  } catch (error) {  
    console.log("Error Liking article:", error);  
  }  
};  
  
const fetchLike = async () => {  
  try {  
    const res = await axios.get<Like>(`articles/${id}/like`);  
    likes.value = res.data.likes;  
  }  
};
```

```
    } catch (error) {  
      console.log("Error fetching likes:", error);  
    }  
  };  
  
  onMounted(fetchLike);
```

由于后端逻辑有变, 需要修改的地方

这一次的 JWT 因为在后端部分就带有 `Bearer` 前缀, 所以前端部分 `axios.ts` 无需再加上 `Bearer` 前缀

否则会出现:

```
Bearer Bearer ...(jwt)
```

的现象, 所以在 `axios.ts` 由

```
config.headers.Authorization = "Bearer " + token;
```

修改为

```
config.headers.Authorization = token;
```

即可。

可修改的地方:

由于这一次的后端逻辑是 `GET articles`, 即为获取所有的文章, 也必须要登录/注册(文章预览页若未登录/注册, 亦无法显示任何内容), 不存在点击查看文章提示登录/注册后再看的逻辑, 所以前端部分可以微小改动, 使逻辑更合理, 当然也可以进一步改动。

将 `NewsView.vue` 中的

```
<div v-else class="no-data">No data</div>
```

改成


```
<div v-else class="no-data">您必须登录/注册才可以阅读文章</div>
```

自然也可以调整后端代码, 维持原先的逻辑。

敲命令:

```
npm run dev
```

Redis 案例, 缓存实战——基础知识 常见的缓存设计模式

为何要有缓存技术?

减轻数据库访问压力, 加快请求响应速度。

缓存读取速度比从数据库中读取快得多。亦可大幅减少数据库查询的次数, 提高应用程序的响应速度。

案例:

文章预览页面, 若有非常多的文章内容(几千条内容), 且此项目用户量较大

若无缓存, 所有的请求都需要直接访问数据库,

可能带来的风险: 应用程序响应变慢, 数据库容易被大量查询拖慢, 影响整个系统的稳定性。

常见的缓存设计模式

1. 旁路缓存模式(Cache-Aside 模式):

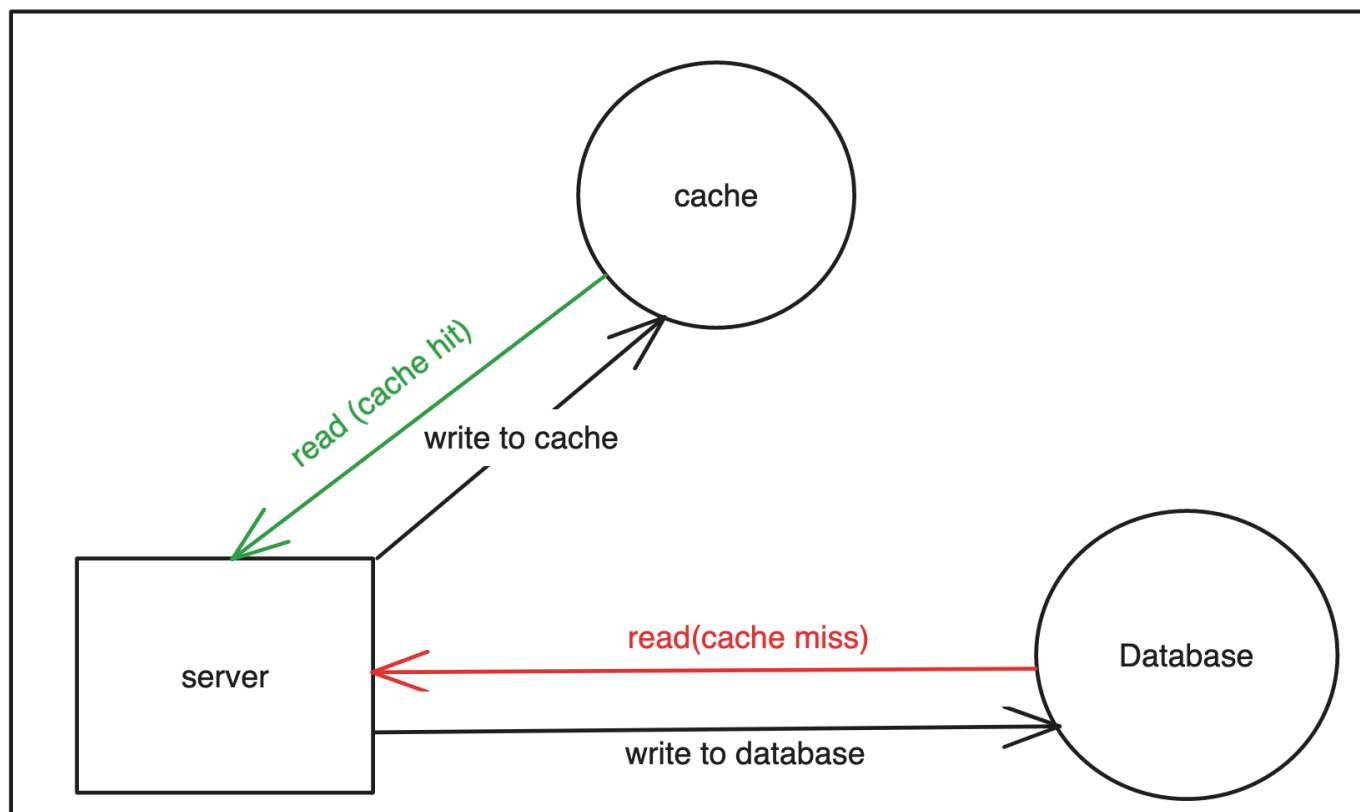
应用程序直接与缓存和数据库交互, 并负责管理缓存的内容。使用该模式的应用程序, 会同时操作缓存与数据库

具体流程如下:

先尝试从缓存中读取数据。若缓存命中, 直接返回缓存中的数据,

若缓存未命中, 则从数据库中读取数据, 并将数据存入缓存中, 然后返回给客户端。

简单概念图:(图源: Yandex)



代码逻辑:

1. 缓存未命中:

- 如果 Redis 中没有找到对应的缓存(缓存未命中), 代码会从数据库中获取文章数据。
- 获取到数据后, 代码将数据序列化为 JSON, 并将其存储在 Redis 缓存中, 同时设置一个过期时间。
- 最后, 返回数据库中的数据给客户端。

2. 缓存命中:

- 如果缓存命中(Redis 中找到了对应的缓存数据), 代码直接从缓存中获取数据。
- 然后, 将缓存中的数据反序列化为文章列表, 返回给客户端。

(在读取数据时, 只有在缓存未命中的情况下, 才会查询数据库并将结果写入缓存。)

此外, 还有如**读写穿透模式**等等模式。

解决下一个问题

若在缓存有效期内,用户又新增了一些文章,此时用户通过缓存得到文章,将看不到变化。

解决方法案例: 常见的缓存失效策略

1. 设置过期时间(我们已经做过了)

2. 主动更新缓存

如: 当新增文章时,除了更新数据库,还要同步更新或删除缓存中的对应数据。这样,下一次读取时,缓存中的数据就是最新的。

或者新增文章时,不直接更新缓存,而是删除缓存中的旧数据。

下次读取时,由于缓存没有命中,从数据库中读取最新数据并重新写入缓存。

Redis `DEL` 命令: 用于删除已存在的键。

CORS

发现问题:

```
✖ Access to XMLHttpRequest at 'http://localhost:3000/api/exchangeRates' from origin 'exchange:1 http://localhost:5173' has been blocked by CORS policy: Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present on the requested resource.
```

CORS 的概念:

CORS, 全称为"跨域(跨源)资源共享"(Cross-Origin Resource Sharing), 是一种机制, 使用额外的 HTTP 头来告诉浏览器允许一个网页从另一个域(不同于该网页所在的域)请求资源。这样可以在服务器和客户端之间进行安全的跨域通信。

浏览器将 CORS 请求分成两类:

- 简单请求(simple req)

方法如 `GET`, `POST`

头部字段如: `Accept`, `Accept-Language`, `Content-Language`, `Content-Type` (需要注意额外的限制)等

在 Postman 中检查 `content-type` 及 `content-length`

✓	Content-Type	i	application/json
✓	Content-Length	i	<calculated when request is sent>

- 非简单请求(not-so-simple req)。

方法如 `PUT`

在正式通信之前, 增加一次 `HTTP` 查询请求, 称为 `预检(Preflight)` 请求

该请求是 `Option` 方法的, 通过该请求来知道服务端是否允许跨域请求。

我们可以在 `Code` 内看到:

```
[GIN] 2024/08/31 - 21:54:01 | 204 | 0s | ::1 | OPTIONS "/api/exchangeRates"
```

对于简单请求, 浏览器直接发出 `CORS` 请求。

CORS 相关的问题

域是什么?

`Origin`, 可译为源, 亦可译为域, 在 `CORS` 上下文中 `Origin` 由三个元素组成:

`Origin` 即为 `协议(如https)` + `域名(如www.example.com)` + `端口(如80)`

同源策略(Same-Origin Policy, `SOP`)

浏览器的一种安全机制, 用于防止恶意网站通过脚本对其他网站的内容进行访问。

以下 `URL` 属于同源地址:

如: `example.com:443` 和 `example.com:443/articles`

但 `inkkaplum频道B站.example.com/articles` 和 `example.com/articles` 则不是。

自然, `example.com:80` 和 `example.com:443` 则不是

跨域请求

指从一个域向另一个域发起的 HTTP 请求。

如从前端应用向不同的后端 API 服务器请求数据, 但是同源策略默认会阻止这些请求。

所以需要 CORS 机制来显式允许跨域访问。

本案例的 URL:

(前端)<http://localhost:5173> 和(后端)<http://localhost:3000>

浏览器默认允许同源请求, 但是默认会阻止这些跨域请求, 除非服务器明确允许。

要解决这个问题, 需在后端应用中配置 CORS, 允许前端应用<http://localhost:5173>访问后端 API。

安装 CORS 中间件

使用 `Gin CORS middleware`, 可以很方便地在 Gin 中配置 CORS。

官方案例:

<https://github.com/gin-contrib/cors>

命令

```
go get github.com/gin-contrib/cors
```

```
package main

import (
    "time"

    "github.com/gin-contrib/cors"
    "github.com/gin-gonic/gin"
)

func main() {
```

```
router := gin.Default()
// CORS for https://foo.com and https://github.com origins,
allowing:
// - PUT and PATCH methods
// - Origin header
// - Credentials share
// - Preflight requests cached for 12 hours
router.Use(cors.New(cors.Config{
    AllowOrigins:    []string{"https://....com"},
    AllowMethods:    []string{"PUT", "PATCH"},
    AllowHeaders:    []string{"Origin"},
    ExposeHeaders:   []string{"Content-Length"},
    AllowCredentials: true,
    AllowOriginFunc: func(origin string) bool {
        return origin == "https://github.com"
    },
    MaxAge: 12 * time.Hour,
}))
router.Run()
```

优雅地退出我们的应用

我们目前可以看到

```
exit status 0xc000013a
```

我们需要进行处理。

参考此文档:

<https://gin-gonic.com/zh-cn/docs/examples/graceful-restart-or-stop/>

Go 语言 Channel

前置知识:

Go 语言的并发模型是 CSP(Communicating Sequential Processes/译为通信顺序进程),

提倡通过通信共享内存而非通过共享内存而实现通信。

Channel 类型: 通道在 Go 中是一种特殊的类型。

通道像一个队列(Queue), 总是遵循先入先出(FIFO)的规则以保证收发数据的顺序。

例子:

```
var channelExample chan int    // 声明一个传递整型的通道
```

判断:

```
chan T    chan<- T    <-chan T
```

通道是引用类型, 故空值为 `nil`。

案例:

```
var channelExample chan int
fmt.Println(channelExample) //输出<nil>
```

声明的 Channel, 需用 make func 初始化之后才能使用。格式如下:

```
make(chan 元素类型, [缓冲(Buffer)大小(可选)])
// 例子:
channelExample01 := make(chan []int)
```

操作:

有发送(Send)、接收(Receive)和关闭(Close)三种操作。

发送的案例:

```
var channelExample chan int

channelExample <- 114
```

接收的案例

```
example := <- channelExample
<-channelExample
```

关闭的案例

通过调用内置的 `close` func 来关闭 channel。

```
close(channelExample)
```

关闭后的通道有如下特点：

- 再发送值就会导致 panic。
- 对其进行接收会一直获取值直到通道为空。
- 对一个关闭的并且没有值的通道执行接收操作会得到对应类型的零值。
- 关闭一个已经关闭的通道会导致 panic。

判断 channel 是否关闭

```
v, ok := <-channel
```

死锁(Deadlock)

```
fatal error: all goroutines are asleep - deadlock!
```

具体案例分析：

第一种情形

```
func main() {
    channel := make(chan int, 0)

    channel <- 114

    x := <- channel

    fmt.Println(x)
}
```


解决方法, 缓冲大小改变, 如 `channel := make(chan int, 1)`

但是:

```
func main() {  
    channel := make(chan int, 1)  
    channel <- 10  
    channel <- 10  
    x := <- channel  
  
    fmt.Println(x)  
}
```

或者,

```
func main() {  
    channel := make(chan int, 0)  
  
    go func(){  
        channel <- 114  
    }()  
    x := <- channel  
  
    fmt.Println(x)  
}
```

第二种情形

```
func main() {  
    channel := make(chan int)  
    x := <- channel  
    go func() {  
        channel <- 114  
    }()  
    fmt.Println(x)  
}
```

以及:

第三种情形

```
func main() {
channel01 := make(chan int)
channel02 := make(chan int)

go func() {
    select {
    case <- channel01:
        channel02<-114
    }
}()

    select {
    case <- channel02:
        channel01 <- 114
    }
}

//结论: fatal error: all goroutines are asleep - deadlock!
```

Select 说明:

select 是 Go 中的一个控制结构。

官方文档关闭案例

<https://gin-gonic.com/zh-cn/docs/examples/graceful-restart-or-stop/>

```
// +build go1.8

package main

import (
    "context"
    "log"
    "net/http"
    "os"
    "os/signal"
    "time"

    "github.com/gin-gonic/gin"
```

```

)

func main() {
    router := gin.Default()
    router.GET("/", func(c *gin.Context) {
        time.Sleep(5 * time.Second)
        c.String(http.StatusOK, "Welcome Gin Server")
    })

    srv := &http.Server{
        Addr:      ":8080",
        Handler: router,
    }

    go func() {
        if err := srv.ListenAndServe(); err != nil && err !=
http.ErrServerClosed {
            log.Fatalf("listen: %s\n", err)
        }
    }()

    quit := make(chan os.Signal, 1)
    signal.Notify(quit, os.Interrupt)
    <-quit
    log.Println("Shutdown Server ...")

    ctx, cancel := context.WithTimeout(context.Background(),
5*time.Second)
    defer cancel()
    if err := srv.Shutdown(ctx); err != nil {
        log.Fatal("Server Shutdown:", err)
    }
    log.Println("Server exiting")
}

```

然后即可看到改变:

```

2024/08/30 23:34:22 Shutdown Server...
2024/08/30 23:34:22 Server exiting

```

WaitGroup

`WaitGroup` 的重要说明: `WaitGroup` 是 `sync` package 中的一个 `struct`, 用来收集需要等待执行完成的 `goroutine`。

三个方法:

`Add()`: 用来设置或添加要等待完成的 `goroutine` 数量

例子:

`Add(2)` 或两次调用 `Add(1)` 都会设置等待计数器的值为 2, 即为要等待 2 个 `goroutine` 完成。

自然, `Done()` 则表示需要等待的 `goroutine` 在真正完成之前, 应调用该方法来人为表示 `goroutine` 完成了, 重要地是: 该方法会对等待计数器减 1。

最后, `Wait()` 则意味着在等待计数器减为 0 之前, `Wait()` 会一直阻塞当前的 `goroutine`

案例代码:

```
func TestTaskControl(t *testing.T) {
    taskNum := 5

    wg := sync.WaitGroup{}
    wg.Add(taskNum)

    for i:=0; i< taskNum; i++){
        go func(i int){
            fmt.Println("info", i)
            wg.Done()
        }(i)
    }

    wg.Wait()
}
```

结论:

```
info 4  
info 3  
info 1  
info 0  
info 2
```

Close案例

```
func Test(t *testing.T){  
    test := make(chan int, 10)  
  
    go func(info chan int){  
        for{  
            select{  
            case val, ok := <- test:  
            if !ok{  
                t.Logf("Channel Closed!")  
                return  
            }  
  
            t.Logf("data %d\n", val)  
        }  
    }(test)  
  
    go func(){  
        test <- 1  
        time.Sleep(1 * time.Second)  
        test <- 2  
  
        close(test)  
    }()  
  
    time.Sleep(5 * time.Second)  
}
```

输出

```
data 1  
data 2  
Context Closed!
```

单独退出通道

传输数据不共用一个 channel

```
func TestA(t *testing.T) {
    test := make(chan int, 5)
    exit := make(chan struct{})

    go func(info chan int, exit chan struct{}) {
        for {
            select {
            case val := <-info:
                t.Logf("data %d\n", val)

            case <-exit:
                t.Logf("Task Exit!!\n")
                return
            }
        }
    }(test, exit)

    go func() {
        test <- 1
        time.Sleep(1 * time.Second)
        test <- 2
        close(exit)
    }()
    time.Sleep(5 * time.Second)
}
```

超时任务控制

```
func Test(t *testing.T){
    test := make(chan int, 5)

    go func(info chan int){
        for{
            select{
            case val := <- info:
                t.Logf("Data %d\n", val)
            }
        }
    }(test)
```

```
case <- time.After(2 * time.Second):
t.Logf("Time out!\n")
return
}
}
}(test)

go func(){
test <- 1
time.Sleep(2 * time.Second) //>=2
test <- 2
}()

time.Sleep(5 *time.Second)
}
```

结论:

```
main_test.go:98: Data 1
main_test.go:101: Time out!
```

Context 的场景

Context 是用来让多级 Goroutine 实现通信的一种工具, 并发安全。

多级嵌套: 父任务停止, 子任务停止、 控制停止顺序(如 ABCDEFG, 可以让顺序为 EFG, BC...)

`context.Context` 该接口定义了四个需要实现的方法

源码

```
type Context interface {  
  
    Deadline() (deadline time.Time, ok bool)  
  
    Done() <-chan struct{}  
  
    Err() error  
  
    Value(key any) any  
}
```

```
func Background() Context {  
    return backgroundCtx{}  
}  
  
// TODO returns a non-nil, empty [Context]. Code should use  
context.TODO when  
// it's unclear which Context to use or it is not yet available  
(because the  
// surrounding function has not yet been extended to accept a  
Context  
// parameter).  
func TODO() Context {  
    return todoCtx{}  
}
```

```
type backgroundCtx struct{ emptyCtx }  
type todoCtx struct{ emptyCtx }  
type emptyCtx struct{}
```

```
func (emptyCtx) Deadline() (deadline time.Time, ok bool) {  
    return  
}  
  
func (emptyCtx) Done() <-chan struct{} {  
    return nil  
}  
  
func (emptyCtx) Err() error {  
    return nil  
}
```



```
func (emptyCtx) Value(key any) any {  
    return nil  
}
```

通过 `context.WithTimeout()` 设置上下文的超时时间, 在到达超时之后自动结束。

而 `WithDeadline()` 则将设置上下文需要完成的截止时间。

例子:

```
ctx, cancel := context.WithTimeout(context.Background(), 10 *  
time.Millisecond),
```

`WithValue()` 函数

```
func TestContext(t *testing.T){  
    a := context.Background()  
    b := context.WithValue(a, "k1", "val1")  
    c := context.WithValue(b, "key1", "val1")  
    d := context.WithValue(c, "key2", "val2")  
    e := context.WithValue(d, "key3", "val3")  
    f := context.WithValue(e, "key3", "val4")  
    fmt.Printf(" %s\n", f.Value("key3"))  
    // 输出: val4  
}
```

传递取消信号

四个方法中, 我们说明了 `Value` 的重要内容, 而其它都是和取消有一定的关系, 所以我们需要分析。

上下文是可以结束的

`Done()` 确定上下文是否完成

而取消上下文则是最直接的方式, 之前进行了 `context.WithCancel` 已经进行了演示。

基本使用的案例

```
func Test(t *testing.T) {
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

    go func() {
        for {
            select {
            case <-ctx.Done():
                t.Log("Context cancelled!")
                return
            }
        }
    }()

    go func() {
        time.Sleep(1 * time.Second)
        cancel()
    }()

    time.Sleep(2 * time.Second)
}
```

场景描述

我们有一个父任务 **A**，它启动了三个子任务 **B**、**C** 和 **D**。每个子任务还会进一步启动自己的子任务，例如，**B** 启动 **E** 和 **F**，**C** 启动 **G**。我们希望通过使用 `context` 来实现以下功能：

1. 多级嵌套控制
2. 控制停止顺序

顺序 `E->F->B->G->C->D->A`

示例代码

```
package main
```

```
import (
    "context"
    "fmt"
    "sync"
    "time"
)

func task(name string, ctx context.Context, wg *sync.WaitGroup)
{
    defer wg.Done()

    fmt.Printf("Task %s started\n", name)

    for {
        select {
            case <-ctx.Done():
                fmt.Printf("Task %s stopped\n", name)
                return
            default:

                time.Sleep(500 * time.Millisecond)
        }
    }
}

func main() {

    ctxA, cancelA := context.WithCancel(context.Background())

    ctxB, cancelB := context.WithCancel(ctxA)
    ctxC, cancelC := context.WithCancel(ctxA)
    ctxD, _ := context.WithCancel(ctxA)

    ctxE, _ := context.WithCancel(ctxB)
    ctxF, _ := context.WithCancel(ctxB)

    ctxG, _ := context.WithCancel(ctxC)

    wg := sync.WaitGroup{}

    wg.Add(1)
    go task("A", ctxA, &wg)
```

```
wg.Add(1)
go task("B", ctxB, &wg)

wg.Add(1)
go task("C", ctxC, &wg)

wg.Add(1)
go task("D", ctxD, &wg)

wg.Add(1)
go task("E", ctxE, &wg)

wg.Add(1)
go task("F", ctxF, &wg)

wg.Add(1)
go task("G", ctxG, &wg)

time.Sleep(2 * time.Second)

cancelB()
time.Sleep(1 * time.Second)

cancelC()
time.Sleep(1 * time.Second)

cancelA()
time.Sleep(1 * time.Second)

wg.Wait()
fmt.Println("All tasks stopped")
}
```

输出

```
Task B started
Task C started
Task D started
Task E started
Task F started
Task G started
```

```
Task E stopped
Task F stopped
Task B stopped
Task G stopped
Task C stopped
Task D stopped
Task A stopped
All tasks stopped
```

和 gin.Context 的关联

go context 和 `gin.Context` 有一定关联。

源码部分:

```
type Context struct {
    writermem responseWriter
    Request    *http.Request
    Writer     ResponseWriter

    Params    Params
    handlers  HandlersChain
    index     int8
    fullPath string

    engine      *Engine
    params      *Params
    skippedNodes []*skippedNode

    // This mutex protects Keys map.
    mu sync.RWMutex

    // Keys is a key/value pair exclusively for the context of
    // each request.
    Keys map[string]any

    // Errors is a list of errors attached to all the
    // handlers/middlewares who used this context.
    Errors errorMsgs
```

```

    // Accepted defines a list of manually accepted formats for
    content negotiation.
    Accepted []string

    // queryCache caches the query result from
    c.Request.URL.Query().
    queryCache url.Values

    // formCache caches c.Request.PostForm, which contains the
    parsed form data from POST, PATCH,
    // or PUT body parameters.
    formCache url.Values

    // SameSite allows a server to define a cookie attribute
    making it impossible for
    // the browser to send this cookie along with cross-site
    requests.
    sameSite http.SameSite
}

```

继续阅读:

```

func (c *Context) Deadline() (deadline time.Time, ok bool) {
    if !c.hasRequestContext() {
        return
    }
    return c.Request.Context().Deadline()
}

// Done returns nil (chan which will wait forever) when
c.Request has no Context.
func (c *Context) Done() <-chan struct{} {
    if !c.hasRequestContext() {
        return nil
    }
    return c.Request.Context().Done()
}

// Err returns nil when c.Request has no Context.
func (c *Context) Err() error {
    if !c.hasRequestContext() {
        return nil
    }
}

```

```

}
return c.Request.Context().Err()
}

// Value returns the value associated with this context for
// key, or nil
// if no value is associated with key. Successive calls to
// Value with
// the same key returns the same result.
func (c *Context) Value(key any) any {
    if key == ContextRequestKey {
        return c.Request
    }
    if key == ContextKey {
        return c
    }
    if keyAsString, ok := key.(string); ok {
        if val, exists := c.Get(keyAsString); exists {
            return val
        }
    }
    if !c.hasRequestContext() {
        return nil
    }
    return c.Request.Context().Value(key)
}

```

Gin 框架里的 Context 也是对 Context 接口的实现, 并增加了许多其他信息。

Go-redis的提示:

最新版本的客户端在操作redis时, 相关函数需要传递上下文(context.Context)

```
go get github.com/go-redis/redis/v8
```

添加内容

视频提及到的建议修改内容

Yml文件中进行更改

```
redis:
  addr: localhost:6379
  DB: 0
  Password: ""
```

空值 `""` 或留空

如:

```
Password:
```

总结

以上就是全部内容,如果有任何问题,欢迎私信 UP 主反馈!

内容结尾提示

本教程中全部文字版教程和代码为 B 站: [InkkaPlum 频道](#) 和知乎: [Inkka Plum](#) 的相关教程所用,仅供学习。

不得二次用于任何机构/个人再次录制 Go / Gin / Gorm / Redis / MySQL / Vue 或其它任何语言,框架,架构,工具等等教程中。

结语

这是一个比较综合的 Go+Gin+Gorm+Redis+MySQL 教程,Up 顾及到了很多基本概念,因此没有学习过 Go 语言的朋友亦可以学习此内容,但是之后可能需要花一定的时间学习 Go 基础,请观看 Up(B 站 InkkaPlum 频道)的前两期 Go 视频。

<https://www.bilibili.com/video/BV1u142187Ps>

<https://www.bilibili.com/video/BV1kz421i7iB>

此外, 请关注 Up 的 B 站频道和知乎, 并且别忘了一键三连, 当然如果愿意, 欢迎给 Up 充电支持, 您的支持是 Up 前进的动力, 将会鼓励 Up 给各位带来更好的视频。

同时, 所有课件和代码都在 GitHub 上分享, 如果感到有帮助, 请给一个 Star 并关注 Up 的 Github。

扩充内容: 之后还会有一个微服务教程, 敬请期待!

Up B 站 InkkaPlum 频道

<https://space.bilibili.com/290859233>

Up 知乎

<https://www.zhihu.com/people/instead-opt>

Up 掘金

<https://juejin.cn/user/3529872175284560>

Up GitHub

<https://github.com/Slumhee>

以上 祝学习成功!

Inkka Plum